

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import sklearn

!wget "https://drive.google.com/uc?export=download&id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU" -O healthyfime.csv

--2024-04-15 15:48:19-- https://drive.google.com/uc?export=download&id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU
Resolving drive.google.com (drive.google.com)... 172.253.63.101, 172.253.63.100, 172.253.63.139, ...
Connecting to drive.google.com (drive.google.com)|172.253.63.101|:443... connected.
HTTP request sent, awaiting response... 303 See Other
Location: https://drive.usercontent.google.com/download?id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU&export=download [following]
--2024-04-15 15:48:19-- https://drive.usercontent.google.com/download?id=1GVhrh2rH6hUunV4Tf7lQoSfSqov9aPjU&export=down
Resolving drive.usercontent.google.com (drive.usercontent.google.com)... 142.251.163.132, 2607:f8b0:4004:c06::84
Connecting to drive.usercontent.google.com (drive.usercontent.google.com)|142.251.163.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 761835 (744K) [application/octet-stream]
Saving to: 'healthyfime.csv'

healthyfime.csv      100%[=====] 743.98K  --.-KB/s    in 0.03s

2024-04-15 15:48:19 (25.9 MB/s) - 'healthyfime.csv' saved [761835/761835]
```

```
df= pd.read_csv('healthyfime.csv')
df.head()
```

	age	gender	height_cm	weight_kg	body fat_%	diastolic	systolic	gripForce	foi
0	27.0	M	172.3	75.24	21.3	80.0	130.0	54.9	
1	25.0	M	165.0	55.80	15.7	77.0	126.0	36.4	
2	31.0	M	179.6	78.00	20.1	92.0	152.0	44.8	
3	32.0	M	174.5	71.10	18.4	76.0	147.0	41.4	
4	28.0	M	173.8	67.70	17.1	70.0	127.0	43.5	

Next steps: [View recommended plots](#)

Data Description:

Id	Features	Description
01	age	age of the customer
02	gender	gender of the customer (M/F)
03	height_cm	height of the customer in cm
04	weight_kg	weight of the customer in kg
05	body_fat_%	% of fat in customer's body
06	diastolic	diastolic blood pressure, measures the pressure in your arteries when your heart rests between beats
07	systolic	systolic blood pressure, measures the pressure in your arteries when your heart beats
08	gripForce	strength of customer's grip
09	sit and bend forward_cm	to measure flexibility of customer
10	sit-ups counts	count of sit-ups customer can perform
11	broad jump_cm	It is the max jump customer can perform in cm
12	class	Category of customer based on the intensity of workout

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 13393 entries, 0 to 13392
Data columns (total 12 columns):
#   Column              Non-Null Count  Dtype
---  -
0   age                  13393 non-null  float64
1   gender               13393 non-null  object
2   height_cm            13393 non-null  float64
3   weight_kg            13393 non-null  float64
4   body fat_%           13393 non-null  float64
5   diastolic            13393 non-null  float64
6   systolic             13393 non-null  float64
7   gripForce            13393 non-null  float64
8   sit and bend forward_cm 13393 non-null  float64
9   sit-ups counts       13393 non-null  float64
10  broad jump_cm        13393 non-null  float64
11  class                13393 non-null  object
dtypes: float64(10), object(2)
```

```
df.replace({"M":0, "F":1} , inplace = True)
df.head()
```

	age	gender	height_cm	weight_kg	body fat_%	diastolic	systolic	gripForce	foi
0	27.0	0	172.3	75.24	21.3	80.0	130.0	54.9	
1	25.0	0	165.0	55.80	15.7	77.0	126.0	36.4	
2	31.0	0	179.6	78.00	20.1	92.0	152.0	44.8	
3	32.0	0	174.5	71.10	18.4	76.0	147.0	41.4	
4	28.0	0	173.8	67.70	17.1	70.0	127.0	43.5	

Next steps:

 [View recommended plots](#)

```
classes = list(df['class'].unique())
mapping_dict = { ch : i for i, ch in enumerate(sorted(classes, reverse=True)) }
print (mapping_dict)
df['class'].replace(mapping_dict , inplace = True)
df.head()
```

{'D': 0, 'C': 1, 'B': 2, 'A': 3}

	age	gender	height_cm	weight_kg	body fat_%	diastolic	systolic	gripForce	foi
0	27.0	0	172.3	75.24	21.3	80.0	130.0	54.9	
1	25.0	0	165.0	55.80	15.7	77.0	126.0	36.4	
2	31.0	0	179.6	78.00	20.1	92.0	152.0	44.8	
3	32.0	0	174.5	71.10	18.4	76.0	147.0	41.4	
4	28.0	0	173.8	67.70	17.1	70.0	127.0	43.5	

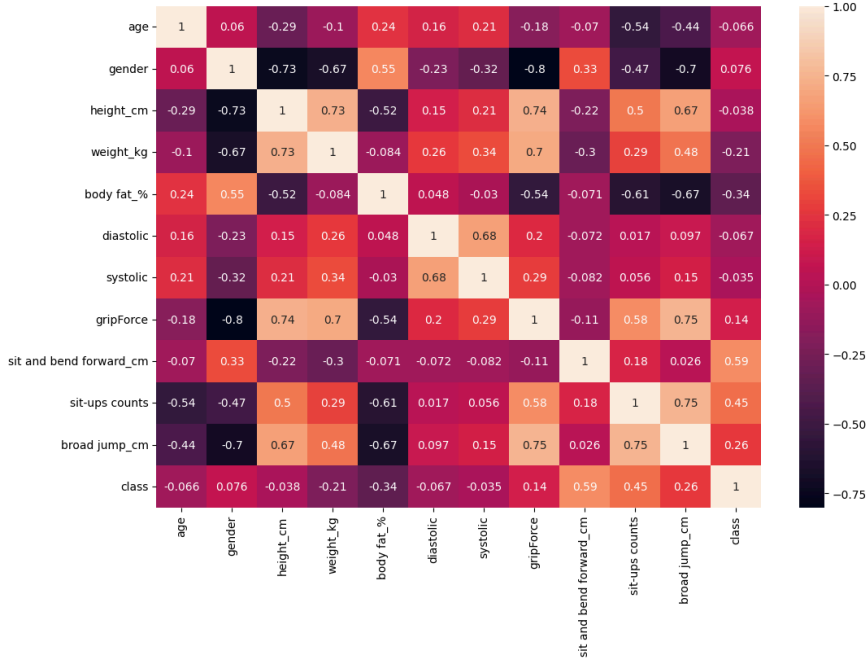
Next steps:

 [View recommended plots](#)

```
plt.figure(figsize=(12,8))

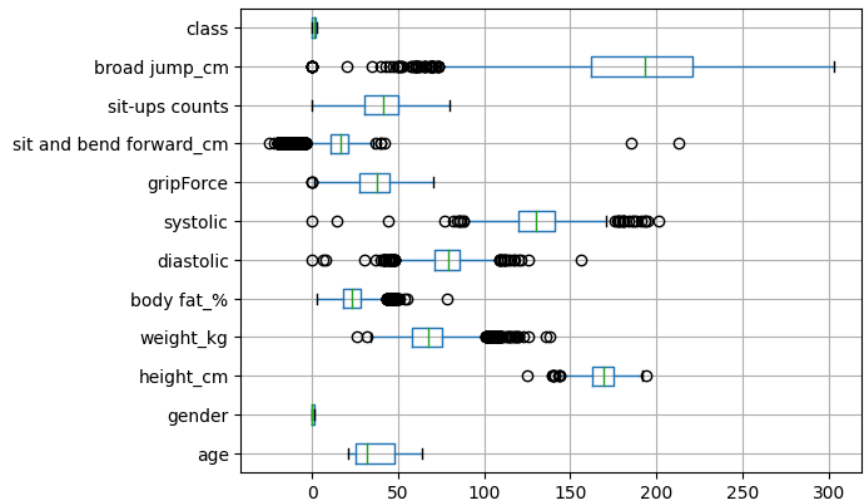
sns.heatmap(df.corr(), annot=True)
```

<Axes: >



```
df.boxplot(rot = 0, vert=False)
```

<Axes: >



```
X, y = df.iloc[:, :-1], df.iloc[:, -1]
print(X.shape, y.shape)

(13393, 11) (13393,)
```

```

from sklearn.model_selection import train_test_split

X_dev, X_test, y_dev, y_test = train_test_split(X, y, test_size=0.1, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_dev, y_dev, test_size=0.1, random_state=42)

print('Train : ', X_train.shape, y_train.shape)
print('Valid : ', X_val.shape, y_val.shape)
print('Test : ', X_test.shape, y_test.shape)

Train : (10847, 11) (10847,)
Valid : (1206, 11) (1206,)
Test : (1340, 11) (1340,)

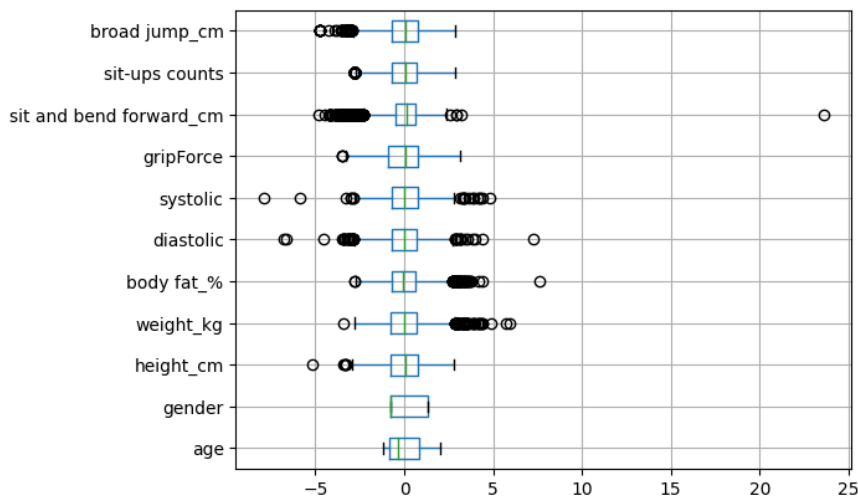
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train,y_train)
X_val = scaler.transform(X_val)
X_test = scaler.transform(X_test)

pd.DataFrame(X_train,columns=df.columns[:-1]).boxplot(rot = 0, vert=False)

```

<Axes: >



```
import tensorflow as tf
```

```
tf.__version__
'2.15.0'
```

```
dir(tf)
```

```
dir(tf.keras)
```

```

['Input',
 'Model',
 'Sequential',
 '__builtins__',
 '__cached__',
 '__doc__',
 '__file__',
 '__internal__',
 '__loader__',
 '__name__',
 '__package__',
 '__path__',
 '__spec__',
 'activations',
 'applications',
 'backend',
 'callbacks',
 'constraints',
 'datasets',
 'dtensor',
 'estimator',
 'experimental',
 'export',
 'initializers',
 'layers',

```

```
'losses',
'metrics',
'mixed_precision',
'models',
'optimizers',
'preprocessing',
'regularizers',
'saving',
'utils']
```

```
dir(tf.keras.activations)
```

```
['__builtins__',
'__cached__',
'__doc__',
'__file__',
'__loader__',
'__name__',
'__package__',
'__path__',
'__spec__',
'deserialize',
'elu',
'exponential',
'gelu',
'get',
'hard_sigmoid',
'linear',
'mish',
'relu',
'selu',
'serialize',
'sigmoid',
'softmax',
'softplus',
'softsign',
'swish',
'tanh']
```

```
dir(tf.keras.models)
```

```
['Model',
'Sequential',
'__builtins__',
'__cached__',
'__doc__',
'__file__',
'__loader__',
'__name__',
'__package__',
'__path__',
'__spec__',
'clone_model',
'experimental',
'load_model',
'model_from_config',
'model_from_json',
'model_from_yaml',
'save_model']
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential([Dense(64, activation='relu', input_shape=(11,)),
                    Dense(4, activation='Softmax')
                    ])
```

```
for i in model.weights:
    print(i.shape)
```

```
(11, 64)
(64,)
(64, 4)
(4,)
```

```
#model = Sequential()
#model.add(Dense(64, activation="relu", input_shape=(11,)))
#model.add(Dense(4, activation="softmax"))
```

```
model.summary()
```

```
Model: "sequential_7"

Layer (type)                 Output Shape                 Param #
=====
dense_6 (Dense)              (None, 64)                  768
dense_7 (Dense)              (None, 4)                   260
=====
Total params: 1028 (4.02 KB)
Trainable params: 1028 (4.02 KB)
Non-trainable params: 0 (0.00 Byte)

model = Sequential([Dense(64, activation='relu', input_shape=(11,),name="Hidden1"),
                    Dense (4, activation='Softmax', name='Output')
                    ])

model.summary()
```

Model: "sequential_8"

Layer (type)	Output Shape	Param #
Hidden1 (Dense)	(None, 64)	768
Output (Dense)	(None, 4)	260

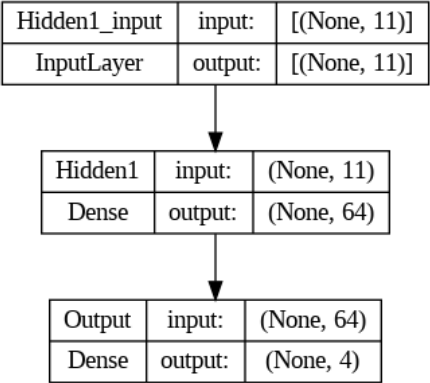
=====

Total params: 1028 (4.02 KB)
Trainable params: 1028 (4.02 KB)
Non-trainable params: 0 (0.00 Byte)

```
#output

from tensorflow.keras.utils import plot_model

plot_model(model,
            to_file='model.png',
            show_shapes=True, show_layer_names=True)
```



```
model.compile(
    loss="sparse_categorical_crossentropy",
    metrics=['accuracy'],
    #optimizer='adam'
    #
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01)
)

dir(tf.keras.optimizers.Adam)
```

```

_keras_api_names_v1,
'lookup_dependency',
'_maybe_initialize_trackable',
'_name_based_attribute_restore',
'_name_based_restores',
'_no_dependency',
'_object_identifier',
'_overwrite_model_variables_with_average_value',
'_preload_simple_restoration',
'_process_kwargs',
'_restore_from_tensors',
'_serialize_hyperparameter',
'_serialize_to_proto',
'_serialize_to_tensors',
'_setattr_tracking',
'_tf_api_names',
'_tf_api_names_v1',
'_track_trackable',
'_trackable_children',
'_unconditional_checkpoint_dependencies',
'_unconditional_dependency_names',
'_update_model_variables_moving_average',
'_update_step',
'_update_step_xla',
'_update_uid',
'_use_weight_decay',
'_var_key',
'add_variable',
'add_variable_from_reference',
'aggregate_gradients',
'apply_gradients',
'build',
'compute_gradients',
'exclude_from_weight_decay',
'finalize_variable_values',
'from_config',
'get_config',
'iterations',
'learning_rate',
'load_own_variables',
'lr',
'minimize',
'save_own_variables',
'set_weights',
'update_step',
'variables']

```

```
model.fit(X_train, y_train, epochs= 10, validation_split=0.1, verbose=1, batch_size=256)
```

```

Epoch 1/10
39/39 [=====] - 1s 10ms/step - loss: 1.0563 - accuracy: 0.5252 - val_loss: 0.8785 - val_accuracy: 0.5252
Epoch 2/10
39/39 [=====] - 0s 3ms/step - loss: 0.8454 - accuracy: 0.6255 - val_loss: 0.8241 - val_accuracy: 0.6255
Epoch 3/10
39/39 [=====] - 0s 3ms/step - loss: 0.8027 - accuracy: 0.6538 - val_loss: 0.8102 - val_accuracy: 0.6538
Epoch 4/10
39/39 [=====] - 0s 3ms/step - loss: 0.7632 - accuracy: 0.6688 - val_loss: 0.7554 - val_accuracy: 0.6688
Epoch 5/10
39/39 [=====] - 0s 4ms/step - loss: 0.7393 - accuracy: 0.6841 - val_loss: 0.7439 - val_accuracy: 0.6841
Epoch 6/10
39/39 [=====] - 0s 3ms/step - loss: 0.7126 - accuracy: 0.7042 - val_loss: 0.7061 - val_accuracy: 0.7042
Epoch 7/10
39/39 [=====] - 0s 3ms/step - loss: 0.7007 - accuracy: 0.7133 - val_loss: 0.7119 - val_accuracy: 0.7133
Epoch 8/10
39/39 [=====] - 0s 4ms/step - loss: 0.6952 - accuracy: 0.7089 - val_loss: 0.6935 - val_accuracy: 0.7089
Epoch 9/10
39/39 [=====] - 0s 3ms/step - loss: 0.6798 - accuracy: 0.7189 - val_loss: 0.6802 - val_accuracy: 0.7189
Epoch 10/10
39/39 [=====] - 0s 4ms/step - loss: 0.6720 - accuracy: 0.7226 - val_loss: 0.6804 - val_accuracy: 0.7226
<keras.src.callbacks.History at 0x7aa45bf57e50>

```

```
model.fit(X_train, y_train, epochs= 10, validation_split=0.1, verbose=1, batch_size=256)
```

```

Epoch 1/10
39/39 [=====] - 0s 9ms/step - loss: 0.6639 - accuracy: 0.7286 - val_loss: 0.6675 - val_accuracy: 0.7286
Epoch 2/10
39/39 [=====] - 0s 11ms/step - loss: 0.6581 - accuracy: 0.7267 - val_loss: 0.6702 - val_accuracy: 0.7267
Epoch 3/10
39/39 [=====] - 0s 10ms/step - loss: 0.6550 - accuracy: 0.7286 - val_loss: 0.6571 - val_accuracy: 0.7286
Epoch 4/10
39/39 [=====] - 0s 9ms/step - loss: 0.6516 - accuracy: 0.7307 - val_loss: 0.6690 - val_accuracy: 0.7307
Epoch 5/10
39/39 [=====] - 0s 8ms/step - loss: 0.6417 - accuracy: 0.7346 - val_loss: 0.6552 - val_accuracy: 0.7346
Epoch 6/10
39/39 [=====] - 0s 10ms/step - loss: 0.6383 - accuracy: 0.7346 - val_loss: 0.6549 - val_accuracy: 0.7346
Epoch 7/10
39/39 [=====] - 0s 10ms/step - loss: 0.6399 - accuracy: 0.7358 - val_loss: 0.6520 - val_accuracy: 0.7358
Epoch 8/10
39/39 [=====] - 0s 10ms/step - loss: 0.6338 - accuracy: 0.7410 - val_loss: 0.6533 - val_accuracy: 0.7410

```

```
Epoch 9/10
39/39 [=====] - 0s 6ms/step - loss: 0.6312 - accuracy: 0.7375 - val_loss: 0.6525 - val_accuracy: 0.7369
Epoch 10/10
39/39 [=====] - 0s 7ms/step - loss: 0.6295 - accuracy: 0.7369 - val_loss: 0.6455 - val_accuracy: 0.7369
<keras.src.callbacks.History at 0x7aa4588d8b80>
```

```
history= model.fit(X_train, y_train, epochs= 100, validation_split=0.1, verbose=1, batch_size=256)
```

```
h 72/100
39 [=====] - 0s 3ms/step - loss: 0.5764 - accuracy: 0.7594 - val_loss: 0.6236 - val_accuracy: 0.7369
h 73/100
39 [=====] - 0s 3ms/step - loss: 0.5784 - accuracy: 0.7613 - val_loss: 0.6250 - val_accuracy: 0.7369
h 74/100
39 [=====] - 0s 3ms/step - loss: 0.5786 - accuracy: 0.7633 - val_loss: 0.6198 - val_accuracy: 0.7369
h 75/100
39 [=====] - 0s 3ms/step - loss: 0.5810 - accuracy: 0.7574 - val_loss: 0.6227 - val_accuracy: 0.7369
h 76/100
39 [=====] - 0s 3ms/step - loss: 0.5750 - accuracy: 0.7608 - val_loss: 0.6223 - val_accuracy: 0.7369
h 77/100
39 [=====] - 0s 3ms/step - loss: 0.5760 - accuracy: 0.7612 - val_loss: 0.6243 - val_accuracy: 0.7369
h 78/100
39 [=====] - 0s 3ms/step - loss: 0.5742 - accuracy: 0.7642 - val_loss: 0.6253 - val_accuracy: 0.7369
h 79/100
39 [=====] - 0s 3ms/step - loss: 0.5772 - accuracy: 0.7633 - val_loss: 0.6232 - val_accuracy: 0.7369
h 80/100
39 [=====] - 0s 3ms/step - loss: 0.5755 - accuracy: 0.7589 - val_loss: 0.6274 - val_accuracy: 0.7369
h 81/100
39 [=====] - 0s 3ms/step - loss: 0.5732 - accuracy: 0.7629 - val_loss: 0.6333 - val_accuracy: 0.7369
h 82/100
39 [=====] - 0s 3ms/step - loss: 0.5807 - accuracy: 0.7608 - val_loss: 0.6475 - val_accuracy: 0.7369
h 83/100
39 [=====] - 0s 3ms/step - loss: 0.5714 - accuracy: 0.7643 - val_loss: 0.6187 - val_accuracy: 0.7369
h 84/100
39 [=====] - 0s 3ms/step - loss: 0.5769 - accuracy: 0.7598 - val_loss: 0.6193 - val_accuracy: 0.7369
h 85/100
39 [=====] - 0s 4ms/step - loss: 0.5746 - accuracy: 0.7617 - val_loss: 0.6176 - val_accuracy: 0.7369
h 86/100
39 [=====] - 0s 3ms/step - loss: 0.5818 - accuracy: 0.7595 - val_loss: 0.6331 - val_accuracy: 0.7369
h 87/100
39 [=====] - 0s 4ms/step - loss: 0.5762 - accuracy: 0.7616 - val_loss: 0.6335 - val_accuracy: 0.7369
h 88/100
39 [=====] - 0s 4ms/step - loss: 0.5720 - accuracy: 0.7609 - val_loss: 0.6466 - val_accuracy: 0.7369
h 89/100
39 [=====] - 0s 3ms/step - loss: 0.5779 - accuracy: 0.7589 - val_loss: 0.6181 - val_accuracy: 0.7369
h 90/100
39 [=====] - 0s 5ms/step - loss: 0.5787 - accuracy: 0.7588 - val_loss: 0.6336 - val_accuracy: 0.7369
h 91/100
39 [=====] - 0s 4ms/step - loss: 0.5739 - accuracy: 0.7637 - val_loss: 0.6234 - val_accuracy: 0.7369
h 92/100
39 [=====] - 0s 4ms/step - loss: 0.5747 - accuracy: 0.7630 - val_loss: 0.6266 - val_accuracy: 0.7369
h 93/100
39 [=====] - 0s 5ms/step - loss: 0.5712 - accuracy: 0.7649 - val_loss: 0.6271 - val_accuracy: 0.7369
h 94/100
39 [=====] - 0s 4ms/step - loss: 0.5722 - accuracy: 0.7622 - val_loss: 0.6375 - val_accuracy: 0.7369
h 95/100
39 [=====] - 0s 4ms/step - loss: 0.5654 - accuracy: 0.7645 - val_loss: 0.6205 - val_accuracy: 0.7369
h 96/100
39 [=====] - 0s 4ms/step - loss: 0.5676 - accuracy: 0.7652 - val_loss: 0.6159 - val_accuracy: 0.7369
h 97/100
39 [=====] - 0s 5ms/step - loss: 0.5706 - accuracy: 0.7644 - val_loss: 0.6354 - val_accuracy: 0.7369
h 98/100
39 [=====] - 0s 5ms/step - loss: 0.5690 - accuracy: 0.7648 - val_loss: 0.6145 - val_accuracy: 0.7369
h 99/100
39 [=====] - 0s 6ms/step - loss: 0.5670 - accuracy: 0.7681 - val_loss: 0.6260 - val_accuracy: 0.7369
h 100/100
39 [=====] - 0s 6ms/step - loss: 0.5696 - accuracy: 0.7636 - val_loss: 0.6366 - val_accuracy: 0.7369
```

```
history.__dict__.keys()
```

```
dict_keys(['validation_data', 'model', '_chief_worker_only', '_supports_tf_logs', 'history', 'params', 'epoch'])
```

```
history.history.keys()
```

```
dict_keys(['loss', 'accuracy', 'val_loss', 'val_accuracy'])
```

```
epochs = history.epoch
loss = history.history["loss"]
accuracy = history.history["accuracy"]
val_loss = history.history["val_loss"]
val_accuracy = history.history["val_accuracy"]
```

```
plt.figure()
plt.plot(epochs, loss, label="train")
plt.plot(epochs, val_loss, label="val")
plt.legend()
```



```
plt.title("Loss VS Epochs")  
plt.show()
```

```
plt.figure()  
plt.plot(epochs, accuracy, label="train")  
plt.plot(epochs, val_accuracy, label="validation")  
plt.legend()  
plt.title("Accuracy VS Epochs")  
plt.show()
```

